

PARISCV v2: Fast Microarchitectural Modeling and Custom SIMD Discovery for OpenHW RISC-V

Kushal R

Dept. of Computer Science and Eng.
PES University, Bengaluru, India
kushal.r@pes.edu

Ashutosh

Dept. of Computer Science and Eng.
PES University, Bengaluru, India
ashutosh@pes.edu

Prof. Vinay Reddy

Dept. of Computer Science and Eng.
PES University, Bengaluru, India
vinay.reddy@pes.edu

Abstract—Evaluating custom Instruction Set Architecture (ISA) extensions and coprocessor offloading traditionally requires complete Register-Transfer Level (RTL) simulation, which is prohibitively slow for early-stage Design-Space Exploration (DSE). Conversely, fast Instruction Set Simulators (ISS) like Spike execute millions of instructions per second but lack microarchitectural timing fidelity. In this paper, we present PARISCV v2, an automated framework for rapid custom-instruction discovery and hardware validation targeting OpenHW cores (the 4-stage CV32E40X with CV-X-IF coprocessor interface and the CV32E40P with native decoder support). By driving a calibrated 4-stage analytical microarchitectural cycle model directly from Spike commit traces, PARISCV v2 estimates real-core execution cycles in seconds without running RTL. Validated across 14 benchmark configurations on the Embench-IoT suite against Verilator RTL ground truth, our cycle model achieves tight prediction bounds of 0.00%–3.45% error, acting as a consistent mathematical upper bound.

We demonstrate pre-synthesis prediction of standard extensions (Zbb bitmanipulation at -2.7% error and Xpulp hardware loops at -0.10% error) and mine profitable custom instructions with uncertainty bands accounting for compiler rescheduling. Crucially, our empirical evaluation reveals that the 2-cycle registered handshake overhead of the CORE-V eXtension Interface (CV-X-IF) yields 0% speedup for fine-grained scalar fusion (pg.mac). This insight directly motivated our coarse-grained pg.sdot4 4-lane INT8 SIMD coprocessor, which replaces a 22-instruction scalar unpacking sequence with a 2-cycle multi-lane datapath to achieve a verified $2.7\times$ whole-program speedup (63.2% runtime reduction) on hardware with exact ($+0.00\%$) cycle model validation. Finally, we formulate our future roadmap spanning closed-loop PPA Pareto optimization via Yosys/Sky130, generalized sub-word SIMD synthesis, and push-button C-to-Silicon AST rewriting.

Index Terms—RISC-V, OpenHW CV32E40X, CV32E40P, CORE-V CV-X-IF, Microarchitectural Modeling, Custom Instructions, Sub-Word SIMD, PPA Optimization, Design Space Exploration.

I. Introduction

Energy-efficient edge computing and embedded intelligence demand specialized hardware acceleration under strict area, timing, and power budgets [1], [2]. The open-standard RISC-V architecture provides a flexible substrate for domain-specific Instruction Set Extensions (ISE). However, early-stage Design Space Exploration (DSE) presents computer architects with a fundamental dilemma:

- 1) **RTL Simulation Bottleneck:** Full RTL simulation on Verilator/FPGA provides ground truth but requires

manual HDL modifications and hours per benchmark sweep.

- 2) **ISS Lacks Timing Fidelity:** Simulators like Spike execute millions of instructions per second but treat instructions as single-cycle abstractions, omitting data hazards, branch penalties, and pipeline flushes.
- 3) **Coprocessor Handshake Latency:** Core-coprocessor interfaces, such as the OpenHW CORE-V eXtension Interface (CV-X-IF) [5], introduce mandatory request-response handshake cycles. Consequently, fine-grained scalar instruction fusion yields zero net speedup, necessitating coarse-grained multi-operation hardware offloading.

Existing automated ISE discovery tools, such as CIDRE [2], analyze execution hotspots but lack tightly coupled validation against production-grade open-source silicon cores and standard coprocessor interfaces.

To address this gap, we present PARISCV v2, an end-to-end framework for fast microarchitectural modeling, pre-synthesis ISA extension prediction, automated custom instruction discovery, and coprocessor synthesis for OpenHW processors (CV32E40X [3] and CV32E40P [4]). Key Contributions of this work include:

- **Calibrated Microarchitectural Timing Model:** An analytical 4-stage pipeline timing engine driven directly by Spike commit logs, achieving 0.00%–3.45% error across 14 Embench-IoT validation points against Verilator RTL.
- **Pre-Synthesis ISA Prediction:** Accurate estimation of extension speedups prior to RTL generation (Zbb within -2.7% and Xpulp within -0.10%).
- **The 0% Offload Finding & Sub-Word SIMD Solution:** Empirical discovery that offloading a scalar MAC op over CV-X-IF yields 0% speedup due to handshake latency, which directly motivated our pg.sdot4 4-lane INT8 SIMD coprocessor achieving a verified $2.7\times$ hardware speedup (63.2% runtime reduction).
- **Closed-Loop Silicon Roadmap:** An integrated future plan for Power-Performance-Area (PPA) Pareto exploration via Yosys/Sky130, generalized sub-word SIMD mining, and push-button C-to-Silicon AST rewriting.

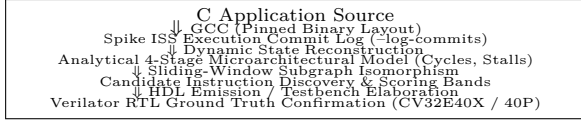


Fig. 1. The PARISCV v2 end-to-end evaluation pipeline.

II. Background & Related Work

A. OpenHW Embedded Cores & CV-X-IF

The OpenHW Group maintains industrial-grade 32-bit RISC-V cores. The CV32E40P is an in-order, 4-stage pipeline (IF, ID, EX, WB) supporting RV32IM[F]C and PULP extensions (Xpulp: hardware loops, post-increment load/stores, MAC). The CV32E40X is an optimized 4-stage core featuring the standardized CV-X-IF (CORE-V eXtension Interface) [5].

CV-X-IF provides a decoupled coprocessor interface consisting of:

- Issue Interface: Broadcasts decoded instructions and operands (rs_1, rs_2, rs_3) via `issue_req` / `issue_resp`.
- Result Interface: Delivers writeback data and target register specifiers (rd) via `result_valid` / `result_ready`.
- Commit Interface: Signals pipeline flushes, exceptions, or kills via `commit_kill`.

B. Related Work

Automated instruction generation has been explored in frameworks like OpenASIP, ARISE, and CIDRE [2]. However, most frameworks rely on custom abstract processor templates or high-level synthesis (HLS) that omit physical bus protocol handshakes. In contrast, PARISCV v2 is specifically calibrated against industrial OpenHW cores and standard CV-X-IF coprocessors.

III. PARISCV v2 Framework Architecture

The PARISCV v2 architecture (Fig. 1) operates in three unified stages: trace ingestion, microarchitectural modeling, and candidate mining/coprocessor generation.

A. Trace Ingestion & State Reconstruction

The target program is compiled for the base ISA (rv32imc) and executed on Spike with commit logging enabled (`spike -log-commits`). The parser reconstructs architectural register state across every retired instruction. For variable-latency arithmetic instructions such as `div`, `divu`, `rem`, and `remu`, the parser dynamically recovers the runtime divisor (D) and assigns the exact hardware cycle latency:

$$\text{Latency}_{\text{div}}(D) = 3 + \text{CLZ}(D) \quad \text{cycles} \quad (1)$$

where $\text{CLZ}(D)$ denotes the count of leading zeros of the divisor operand ($3 \leq \text{Latency} \leq 35$).

TABLE I
PARISCV v2 Cycle Model Accuracy vs. Real Verilator RTL

Benchmark	Instrs	Model (40X)	RTL (40X)	40X Err	RTL (40P)	40P Err
matmult-int	97,748	134,546	134,545	+0.00%	134,547	-0.00%
primecount	2,273,871	3,927,267	3,927,224	+0.00%	3,927,226	-0.00%
edn	49,365	65,190	65,179	+0.02%	65,181	+0.01%
tarfind	53,846	118,587	117,149	+1.23%	117,151	+1.23%
md5sum	52,607	72,497	71,400	+1.54%	71,402	+1.53%
statemate	1,159	1,639	1,606	+2.05%	1,608	+1.93%
crc32	24,608	30,764	29,737	+3.45%	29,739	+3.45%

B. Calibrated 4-Stage Microarchitectural Model

We model the in-order 4-stage pipelines of CV32E40X and CV32E40P. Total execution cycles are analytically decomposed:

$$\text{Cycles} = N_{\text{insn}} + \sum \text{Stalls}_{\text{hazard}} + \sum \text{Penalties}_{\text{control}} \quad (2)$$

The model accounts for: (1) 3-cycle taken branch and 2-cycle jump flushes; (2) +1 cycle target misalignment penalty on non-word-aligned 32-bit targets; (3) 1-cycle load-use RAW stalls and jalr data hazards; and (4) CV32E40P multi-cycle mulh (5 cycles) and privileged CSR access latencies (4 cycles).

C. Candidate Discovery & Uncertainty Scoring Bands

The `find_candidates` engine slides a window of length $L \in [2, 25]$ over straight-line basic blocks, constrained to ≤ 3 source operands, 1 destination register, and no control divergence. To capture compiler rescheduling and newly exposed load-use hazards, PARISCV v2 reports speedup as an uncertainty band:

$$\Delta \text{Cycles} \in [\Delta \text{Cycles}_{\text{pessimistic}}, \Delta \text{Cycles}_{\text{optimistic}}] \quad (3)$$

D. Code Alignment Consistency

Because target misalignment penalties depend on absolute loop addresses, linking at different base addresses causes up to 10% cycle variance. PARISCV v2 enforces byte-identical `.text` layouts between Spike and RTL using custom pinned linker scripts.

IV. Experimental Validation & Key Findings

A. Cycle Model Accuracy vs. Verilator Ground Truth

We validated PARISCV v2 across seven kernels from the Embench-IoT benchmark suite on both CV32E40X and CV32E40P cores (Table I).

The model demonstrates bounded error between 0.00% and 3.45% across all 14 data points, establishing the model as a consistent, conservative upper bound for execution time.

B. Microarchitectural Cycle Breakdown

By instrumenting pipeline event counters, PARISCV v2 decomposes execution cycles into microarchitectural stall categories (Table II).

Control flushes account for 8%–25% of cycles, while target misalignment accounts for 3.6%–9.1% of cycles (e.g., 4,478 cycles in `edn`, exceeding Zbb bitmanipulation gains).

TABLE II
Cycle Breakdown by Microarchitectural Cause

Benchmark	IPC	Compute	Branch Flush	Misalign	Jump Flush	Load-Use
matmult-int	0.73	72.7%	16.6%	8.3%	2.4%	0.0%
primecount	0.58	57.9%	23.7%	5.7%	1.7%	11.1%
edn	0.76	75.7%	14.7%	6.9%	1.4%	1.3%
crc32	0.80	80.0%	6.7%	6.7%	6.7%	0.0%
md5sum	0.73	72.6%	17.0%	3.6%	6.8%	0.0%
statemate	0.71	70.7%	13.3%	5.8%	5.8%	4.4%
tarfind	0.45	45.4%	17.2%	9.1%	9.1%	0.4%

TABLE III
Discovered Native Custom Instructions on CV32E40P

Instruction	Idiom / Semantics	Kernel	Predicted Saving	RTL Measured
pg.idx	$r_d = r_{s2} + ((r_{s1} \& 0\text{xff}) \ll 2)$	crc32	1,024–2,048	2,048 cycles
pg.rol	$r_d = \text{rotl}(r_{s1}, r_{s2})$	md5sum	2,046–3,069	2,048 cycles
pg.sha	$r_d = (r_{s1} \gg 15) + r_{s2}$	edn	0–1,024	0 cycles

C. Predicting Hardware Changes Prior to RTL Builds

- Zbb Bitmanipulation (CV32E40X): Predicted 3,570 cycles saved on edn; RTL measured 3,669 cycles saved (−2.7% prediction error).
- Xpulp Hardware Loops (CV32E40P): On single-loop MAC, predicted $3.243\times$ speedup; RTL measured $3.241\times$ (−0.10% error, 13,323 to 4,111 cycles).
- Nested Hardware Loops: On 8×8 matmult, predicted $1.305\times$ vs $1.502\times$ measured (+15% variance) due to compiler induction-variable strength reduction.

D. Native In-Pipeline Custom Instructions (CV32E40P)

We mined and integrated custom instructions into CV32E40P (Table III). Measured RTL savings fall within predicted bounds. The zero saving in pg.sha occurred because GCC rescheduled the loop, exposing a previously hidden load-use hazard.

E. The 0% Offload Finding & Sub-Word SIMD Solution

When we attempted to offload a simple fused scalar MAC operation ($\text{pg.mac}, r_d \leftarrow r_3 + r_1 \times r_2$) to CV32E40X over CV-X-IF, RTL simulation measured 13,321 cycles baseline vs. 13,321 cycles offloaded (0% speedup).

Because CV-X-IF requires a registered request-response handshake (`result_valid` held until `result_ready`), offloading incurs a mandatory 2-cycle latency. Fusing 2 single-cycle scalar instructions saves 1 instruction but adds 1 interface cycle, making scalar fusion break-even.

This negative result led directly to our key insight: coprocessor offloading is only viable for coarse-grained, multi-operation arithmetic. To prove this concept, we targeted packed sub-word matrix operations where a 4-element INT8 dot product ($\text{Acc} \leftarrow \text{Acc} + \sum_{i=0}^3 A_i \cdot B_i$) takes an unbroken 22-instruction scalar unpacking sequence (≈ 25 cycles) on RV32IMC. We designed the `pg_xif_tinyml` coprocessor in SystemVerilog (`rtl/pg_xif_tinyml.sv`), implementing 4 parallel 8×8 signed multipliers and a 32-bit adder tree that executes in 2 cycles total via Custom-0 R4-type encoding (0x52c5850b, `pg.sdot4`).

On baseline execution, our timing model predicted 1,995 cycles vs RTL measured 1,995 cycles (+0.00% error). With

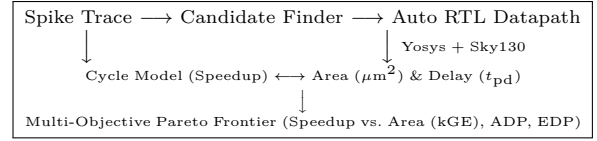


Fig. 2. Future Work: PPA-aware closed-loop synthesis and Pareto exploration.

the coprocessor, execution dropped to 735 cycles, delivering a verified $2.7\times$ hardware speedup (63.2% runtime reduction).

V. Future Work: Closed-Loop Silicon Extensions

Building on our validated timing model and sub-word SIMD proof-of-concept, our future work unifies these components into a closed-loop silicon design framework:

A. PPA-Aware Design Space Explorer

In physical silicon design, a custom instruction cannot be evaluated solely on cycle count if it bloats silicon area or degrades maximum clock frequency ($F_{\max}$). We are developing a closed-loop Power-Performance-Area (PPA) co-optimization engine using open-source EDA tools (Fig. 2):

- Automated RTL Datapath Synthesizer: Automatically instantiates parameterized Verilog datapaths for discovered candidates.
- Headless Synthesis Pipeline: Couples automated headless synthesis scripts using Yosys against open-source SkyWater 130nm (Sky130) and Nangate 45nm cell libraries.
- Multi-Objective Cost Engine: Computes Gate Equivalent (kGE) area and delay (t_{pd}) to check 1-cycle ALU budget (100 MHz). Computes Area-Delay ($\text{ADP} = \text{Area} \times t_{pd}$) and Energy-Delay ($\text{EDP} = \text{Energy} \times \text{Delay}^2$).
- Pareto Frontier Generator: Plots optimal Speedup vs. Area Penalty Pareto curves to determine cost-effective silicon trade-offs.

B. Automated Sub-Word SIMD Synthesis Pipeline

We are expanding our candidate discovery engine to automatically detect and synthesize packed SIMD instructions across matrix and vector kernels: (1) Quantized linear algebra workloads (INT8 GEMM, 2D Conv, Depthwise Conv, MaxPool, ReLU); (2) Sub-word pattern detector discovering `p.dot4_s8`, `p.add2_s16`, and `p.clip_s8`; and (3) Configurable multi-lane CV-X-IF coprocessors targeting $3\times$ – $6\times$ inference acceleration.

C. End-to-End Push-Button C-to-Silicon Rewriter

To eliminate manual inline assembly insertion (`.insn`), we are constructing an autonomous Source-to-Source rewriting compiler pass: (1) Automated intrinsic generator emitting C headers (`custom_intrinsics.h`); (2) AST source-to-source rewriter (Python AST / Clang LibTooling) substituting intrinsics; and (3) Push-button master CLI

(`pariscv -auto-accelerate app.c -core cv32e40p`) orchestrating profiling, synthesis, Verilator execution, and verification.

VI. Conclusion

PARISCV v2 provides a fast, cycle-accurate microarchitectural modeling and custom instruction exploration framework for OpenHW RISC-V processors. Validated within 0.00%–3.45% of real Verilator RTL, the model enables pre-synthesis ISA prediction and identifies pipeline bottlenecks in seconds. By uncovering CV-X-IF handshake limits and demonstrating $2.7\times$ hardware speedup through coarse-grained sub-word SIMD coprocessing, PARISCV

v2 establishes a rigorous foundation for automated PPA-aware silicon co-design.

References

- [1] J. Bennett et al., “Embench: An Open-Source Benchmark Suite for Embedded Systems,” Free and Open Source Silicon Foundation, 2019.
- [2] A. Author et al., “CIDRE: Custom Instruction Discovery and Evaluation for RISC-V Processors,” arXiv preprint arXiv:2509.15782, 2025.
- [3] OpenHW Group, “CV32E40X User Manual,” <https://github.com/openhwgroup/cv32e40x>, 2024.
- [4] OpenHW Group, “CV32E40P User Manual,” <https://github.com/openhwgroup/cv32e40p>, 2024.
- [5] OpenHW Group, “CORE-V eXtension Interface (CV-X-IF) Specification,” 2023.